



CMPT 295

Unit - Machine-Level Programming

Lecture 17 – Assembly language

Function Call and Stack - Passing Control

Last Lecture

- **Function call conventions (protocol)** describing ...
 - 1) Mechanisms for **passing control**
 - 2) Mechanisms for **passing data**
 - 3) Mechanisms for **managing memory allocation** -> **Stack**
 - **Stack frame** allocated to each executing function on **stack memory segment** and deallocated when function terminates
 - Therefore, **function call** has a **LIFO/FILO** behavior
 - **Memory layout** – various segments: **Stack**, Heap, Data, Text / Shared Libraries
- Function call mechanisms in **x86-64**
 - register **stack pointer**: **%rsp**
 - instructions: **pushq** and **popq**

Today's Menu

- Introduction
 - C program -> assembly code -> machine level code
- Assembly language basics: data, `move` operation
 - Memory addressing modes
- Operation `leaq` and Arithmetic & logical operations
- Conditional Statement – Condition Code + `cmovX`
- Loops
- Function call – **Stack**
 - Overview of Function Call
 - Memory Layout and Stack - x86-64 instructions and registers
 - Passing control
 - Passing data – Calling Conventions
 - Managing local data
 - Recursion
- Array
- Buffer Overflow
- Floating-point operations

Remember: **Compressed view of memory**

Why 8?

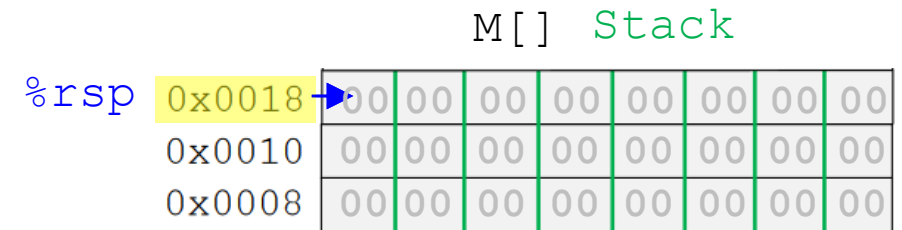
➡ **pushq Src**

1. Get **value** of operand **Src**
2. Decrement **%rsp** by **8 (bytes)**
3. Store **value** at **%rsp**

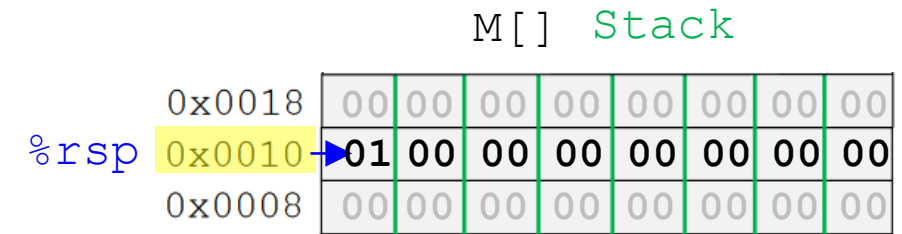
➡ **popq Dest**

1. Read **value** stored at **%rsp**
2. Increment **%rsp** by **8 (bytes)**
3. Load this **value** in operand **Dest**

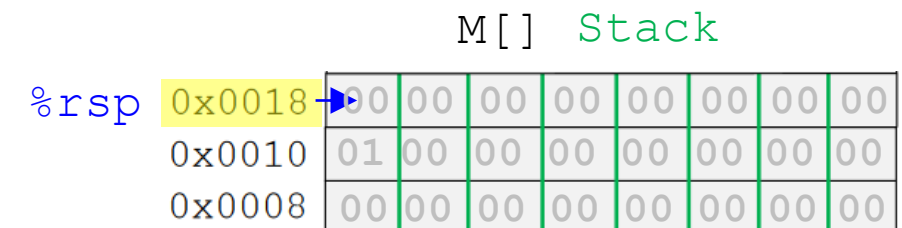
1. **%rsp** contains the memory address 0x0018



- pushq \$1** 2. **%rsp** contains the memory address 0x0010
- 0x0000000000000001



- popq %rdi** 3. **%rsp** contains the memory address 0x0018



1. Control passing mechanism

x86-64 instruction: **call**

➤ Syntax: **call func**

➤ **func** is a label

➤ It is the memory address of the first byte of the first instruction of **func**

Overview of Function Call

What happens when a function (**caller**) calls another function (**callee**) in **x86-64 Assembly Language**?

➔

1. **Control** (execution flow)
2. **Data**
3. **Memory** -> stack memory segment

Looking into the **function call conventions** of **x86-64 Assembly Language**

8

1. Control passing mechanism

x86-64 instruction: **call**

► Let's execute a call: **call func**
and observe the **effect** of executing this **call** on the **stack**

From Slide 9
of Lecture 9

DO FOREVER:

fetch next instruction from memory into CPU and put it into the **instruction register (IR)**

update the **program counter (PC)**

decode the instruction

execute the instruction

```
someFunc:  
400540:some instruction  
400541:some instruction  
400544:call  func  
400549:some instruction  
40054c:some instruction  
40054d:ret
```

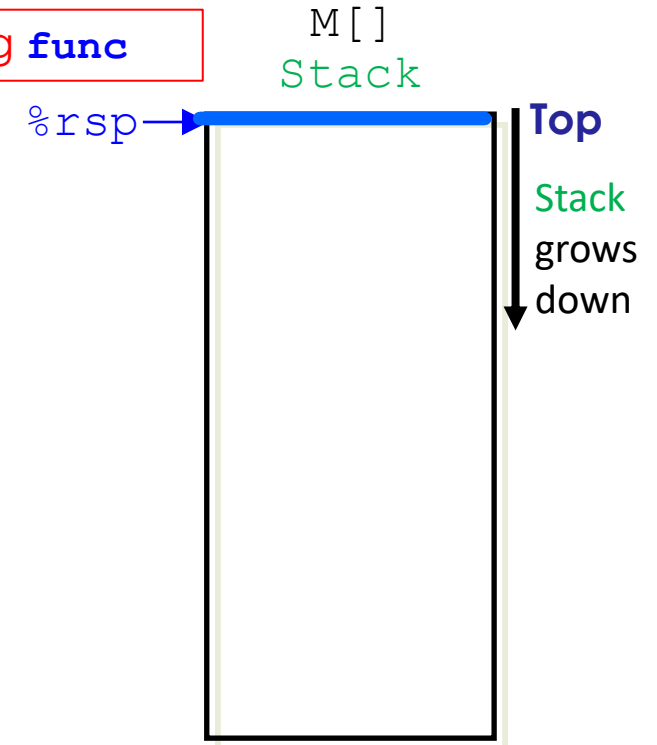
```
func:  
400700:first instruction  
400704:second instruction  
40070A:third instruction  
40070C:ret
```

%rip -> 400544



PC

Before calling **func**



1. Control passing mechanism

x86-64 instruction: **call**

► Let's execute a call: **call func**
and observe the **effect** of executing this **call** on the **stack**

From Slide 9
of Lecture 9

DO FOREVER:

fetch next instruction from memory into CPU and put it into the **instruction register (IR)**

update the **program counter (PC)**

decode the instruction

execute the instruction

```
someFunc:  
400540:some instruction  
400541:some instruction  
400544:call func  
400549:some instruction  
40054c:some instruction  
40054d:ret
```

```
func:  
400700:first instruction  
400704:second instruction  
40070A:third instruction  
40070C:ret
```

%rip -> 400549



PC

Executing the call to **func**

1. **return address**, i.e., the memory address of the instruction after **call func** (held in **%rip**) in assembly program is stored onto the **stack**.
2. **PC (%rip)** is set to the memory address represented by the label **func**

M[]
Stack

Bottom

Top

Stack grows down

1. Control passing mechanism

x86-64 instruction: **call**

► Let's execute a call: **call func**
and observe the **effect** of executing this **call** on the **stack**

From Slide 9
of Lecture 9

DO FOREVER:

fetch next instruction from memory into CPU and put it into the **instruction register (IR)**

update the **program counter (PC)**

decode the instruction

execute the instruction

```
someFunc:  
400540:some instruction  
400541:some instruction  
400544:call  func  
400549:some instruction  
40054c:some instruction  
40054d:ret
```

```
func:  
400700:first instruction  
400704:second instruction  
40070A:third instruction  
40070C:ret
```

%rip -> 400704



PC

After calling **func** ...
(currently executing the
first instruction **%rsp** →
of **func**)

M[]
Stack

Bottom

400549

Top

Stack
grows
down

1. Summary - Control passing mechanism

x86-64 instruction: **call**

➤ Syntax: **call func**

➤ **func** is a label -> memory address of first byte of first instruction of **func**

➤ What happens when **call** executes:

1. Value of program counter (**PC** or in x86-64: **%rip**) saved on the **stack**

➤ **Why?** Because this value is the **return address**, i.e., the memory address of the instruction after the **call func** instruction in the assembly program and this is where the CPU will resume execution once **func** has executed (and returned).

2. Set the program counter (i.e., **%rip**) to memory address of first byte of first instruction of **func**

Microprocessor starts executing **func**

➤ Microprocessor fetches the first instruction of **func**
(its memory address is in **PC**)

DO FOREVER:

fetch next instruction from memory into CPU and put it into the instruction register (**IR**)

update the program counter (**PC**)

decode the instruction

execute the instruction

1. Control passing mechanism

x86-64 instruction: **ret**

➡ Syntax: **ret**

1. Control passing mechanism

x86-64 instruction: **ret**

► Let's execute a return: **ret**
and observe the **effect** of executing this **ret** on the **stack**

From Slide 9
of Lecture 9

DO FOREVER:

fetch next instruction from memory into CPU and put it into the **instruction register (IR)**

update the **program counter (PC)**

decode the instruction

execute the instruction

```
someFunc:  
400540:some instruction  
400541:some instruction  
400544:call  func  
400549:some instruction  
40054c:some instruction  
40054d:ret
```

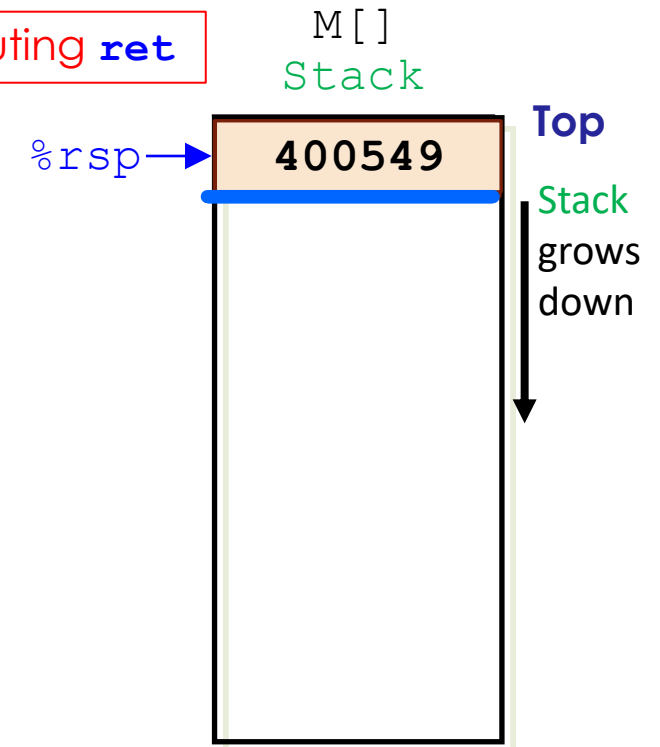
```
func:  
400700:first instruction  
400704:second instruction  
40070A:third instruction  
40070C:ret
```

%rip -> 40070C



PC

Before executing **ret**



1. Control passing mechanism

x86-64 instruction: **ret**

► Let's execute a return: **ret**
and observe the **effect** of executing this **ret** on the **stack**

From Slide 9
of Lecture 9

DO FOREVER:

fetch next instruction from memory into CPU and put it into the **instruction register (IR)**

update the **program counter (PC)**

decode the instruction

execute the instruction

```
someFunc:  
400540:some instruction  
400541:some instruction  
400544:call func  
400549:some instruction  
40054c:some instruction  
40054d:ret
```

```
func:  
400700:first instruction  
400704:second instruction  
40070A:third instruction  
40070C:ret
```

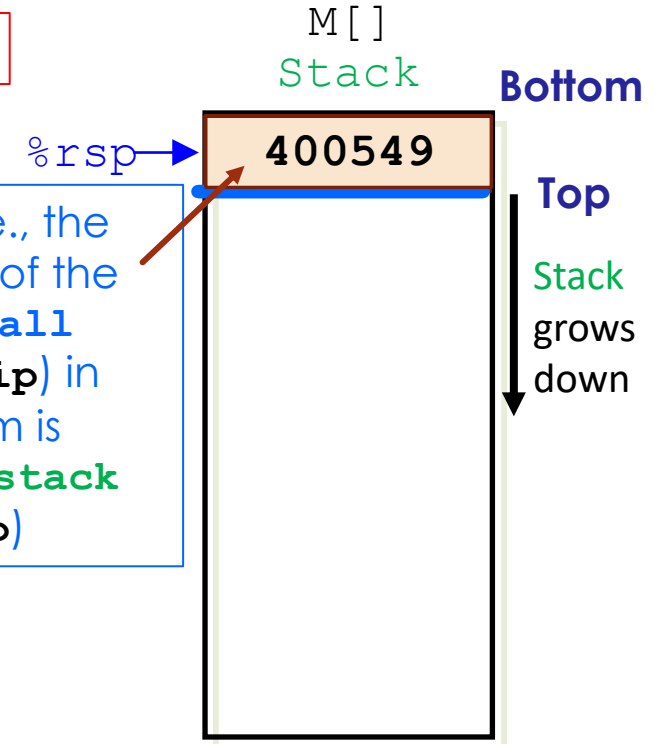
%rip -> 40070D



PC

Executing **ret**

1. **return address**, i.e., the memory address of the instruction after **call func** (held in **%rip**) in assembly program is loaded from the **stack** onto the **PC (%rip)**



1. Control passing mechanism

x86-64 instruction: **ret**

► Let's execute a call: **ret**
and observe the **effect** of executing this **ret** on the **stack**

From Slide 9
of Lecture 9

DO FOREVER:

fetch next instruction from memory into CPU and put it into the **instruction register (IR)**

update the **program counter (PC)**

decode the instruction

execute the instruction

```
someFunc:  
400540:some instruction  
400541:some instruction  
400544:call  func  
400549:some instruction  
40054c:some instruction  
40054d:ret
```

```
func:  
400700:first instruction  
400704:second instruction  
40070A:third instruction  
40070C:ret
```

%rip -> 40054c



PC

After returning from **func** ...
(after executing **%rsp** → **ret**)

M[]
Stack

Top

Stack
grows
down

1. Summary - Control passing mechanism

x86-64 instruction: **ret**

- Syntax: **ret**
- What happens when **ret** executes:
 1. **return address** is loaded (copied) from **stack** onto the **%rip (PC)**

Microprocessor resumes executing **someFunc**

- Microprocessor fetches the instruction at that memory address in **PC**



DO FOREVER:

fetch next instruction from memory into CPU and put it into the *instruction register (IR)*
update the *program counter (PC)*
decode the instruction
execute the instruction

Example

```
void mult_store(long x, long y, long *dest) {  
    long t = mult2(x, y);  
    *dest = t;  
    return;  
}
```

```
long mult2(long a, long b) {  
    long s = a * b;  
    return s;  
}
```

```
0000000000400540 <mult_store>:  
400540: push    %rbx           # Save %rbx  
400541: mov     %rdx,%rbx      # Save dest  
400544: callq   400550 <mult2>  # mult2(x,y)  
400549: mov     %rax, (%rbx)    # Save at dest  
40054c: pop     %rbx           # Restore %rbx  
40054d: retq                      # Return
```

```
0000000000400550 <mult2>:  
400550: mov     %rdi,%rax      # a  
400553: imul    %rsi,%rax      # a * b  
400557: retq                      # Return
```

Example – Steps 1 and 2

x in %rdi,

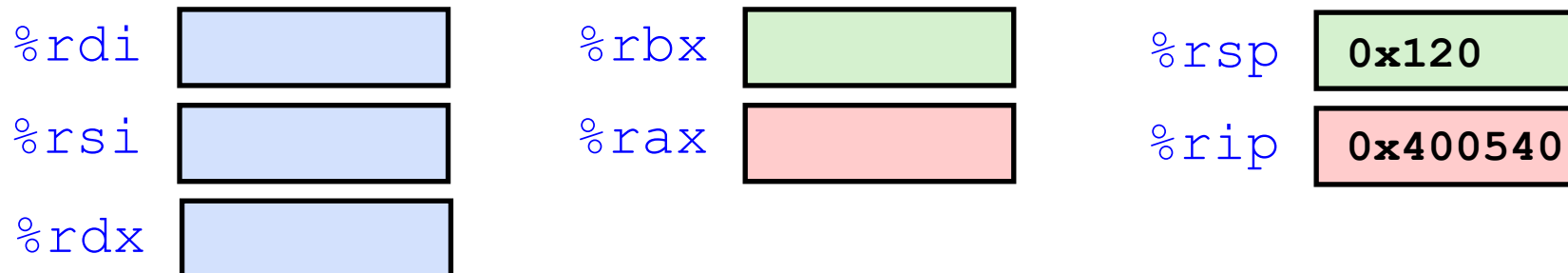
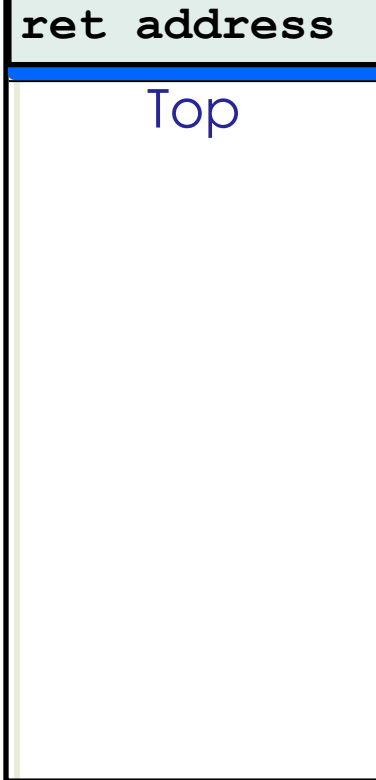
```
0000000000400540 <mult_store>:
1. 400540: push    %rbx          # Save %rbx
2. 400541: mov     %rdx,%rbx      # Save dest
   400544: callq   400550 <mult2>  # mult2(x,y)
   400549: mov     %rax, (%rbx)    # Save at dest
   40054c: pop     %rbx          # Restore %rbx
   40054d: retq                    # Return
```

a in %rdi,

```
0000000000400550 <mult2>:
   400550: mov     %rdi,%rax      # a
   400553: imul    %rsi,%rax      # a * b
   400557: retq                    # Return
```

%rsp →

M[]
Stack

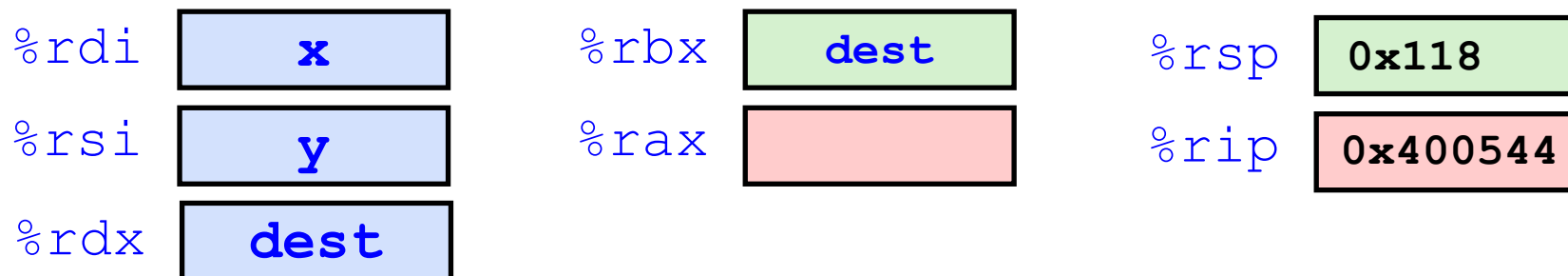


Example – Steps 3 and 4

```
0000000000400540 <mult_store>:
1. 400540: push    %rbx          # Save %rbx
2. 400541: mov     %rdx,%rbx          # Save dest
3. 400544: callq   400550 <mult2>      # mult2(x,y)
   400549: mov     %rax, (%rbx)         # Save at dest
   40054c: pop     %rbx                 # Restore %rbx
   40054d: retq                      # Return
```

```
0000000000400550 <mult2>:
4. 400550: mov     %rdi,%rax           # a
   400553: imul    %rsi,%rax           # a * b
   400557: retq                      # Return
```

`%rsp` →



Example – Steps 5 and 6

```
0000000000400540 <mult_store>:
1. 400540: push    %rbx          # Save %rbx
2. 400541: mov     %rdx,%rbx          # Save dest
3. 400544: callq   400550 <mult2>      # mult2(x,y)
   400549: mov     %rax, (%rbx)        # Save at dest
   40054c: pop     %rbx               # Restore %rbx
   40054d: retq                      # Return
```

```
0000000000400550 <mult2>:
4. 400550: mov     %rdi,%rax          # a
5. 400553: imul    %rsi,%rax          # a * b
6. 400557: retq                      # Return
```

%rdi **x -> a**

%rsi **y -> b**

%rdx **dest**

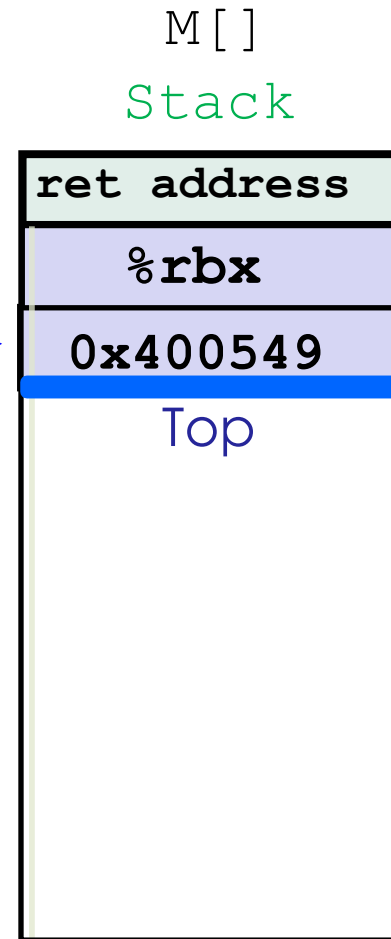
%rbx **dest**

%rax **a**

%rsp **0x110**

%rip **0x400553**

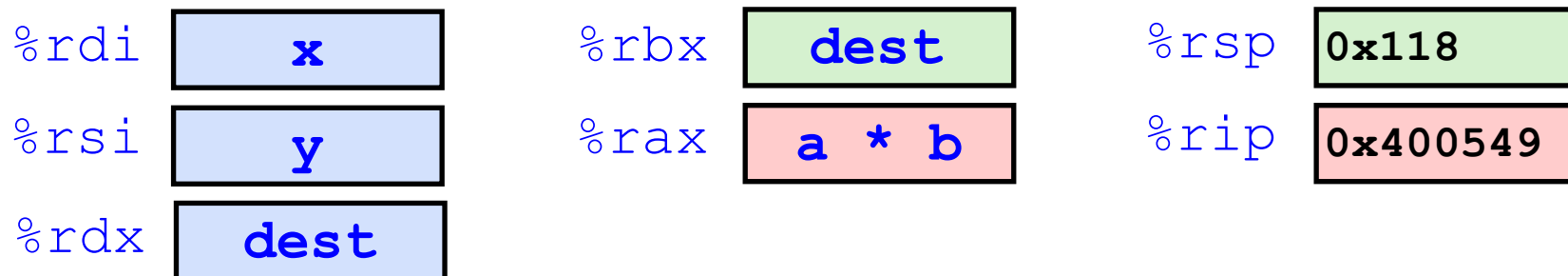
%rsp →



Example – Steps 7, 8 and 9

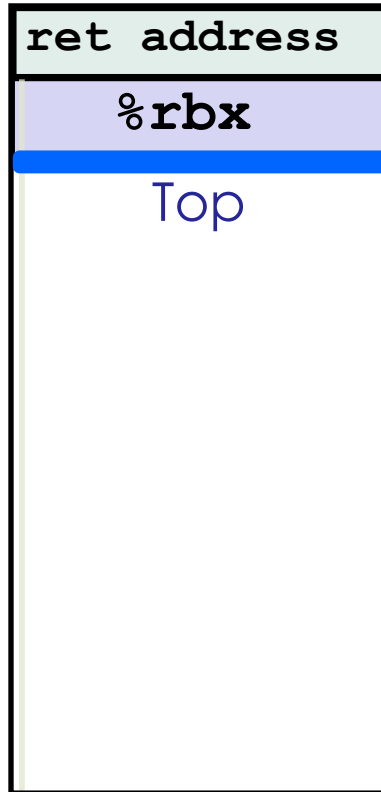
```
0000000000400540 <mult_store>:
1. 400540: push    %rbx          # Save %rbx
2. 400541: mov     %rdx,%rbx        # Save dest
3. 400544: callq   400550 <mult2>    # mult2(x,y)
7. 400549: mov     %rax, (%rbx)      # Save at dest
8. 40054c: pop     %rbx             # Restore %rbx
9. 40054d: retq                      # Return
```

```
0000000000400550 <mult2>:
4. 400550: mov     %rdi,%rax        # a
5. 400553: imul    %rsi,%rax        # a * b
6. 400557: retq                      # Return
```



M[]
Stack

%rsp →



Summary

- **Function call** mechanisms in **x86-64**:
 - 1) Mechanisms for **passing control**
 - **x86-64 stack** register and instructions:
 - Register stack pointer: **%rsp**,
 - Instructions: **pushq** and **popq**
 - **x86-64** function call instructions:
 - **call** and **ret**

Next Lecture

- ▀ Introduction
 - ▀ C program -> assembly code -> machine level code
- ▀ Assembly language basics: data, `move` operation
 - ▀ Memory addressing modes
- ▀ Operation `leaq` and Arithmetic & logical operations
- ▀ Conditional Statement – Condition Code + `cmovX`
- ▀ Loops
- ▀ Function call – **Stack** – Recursion
 - ▀ Overview of Function Call
 - ▀ Memory Layout and Stack - x86-64 instructions and registers
 - ▀ Passing control
 - ▀ Passing data – Calling Conventions
 - ▀ Managing local data
 - ▀ Recursion
- ▀ Array
- ▀ Buffer Overflow
- ▀ Floating-point operations